

IO Systems

— Chapter 13



叶冠华

g.ye@bupt.edu.cn

School of Computer Science (National Pilot Software Engineering School)



北京邮电大学

Mass Storage

- Performance Measures of Disks:
 - ▣ Average I/O time = Seek time + Rotational latency + Data-transfer time (+ Controller overhead)
- Disk Scheduling
 - ▣ FCFS
 - ▣ SSTF
 - ▣ SCAN
 - ▣ Look
 - ▣ C-SCAN
 - ▣ C-Look

Exercise

- 12.2 Suppose that a disk drive has 5000 cylinders, numbered 0 to 4999. The drive is currently serving a request at cylinder 143, and the previous request was at cylinder 125. The queue of pending requests, in FIFO order, is

86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130

Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests, for each of the following disk-scheduling algorithms?

- a. FCFS
- b. SSTF
- c. SCAN
- d. LOOK
- e. C-SCAN

e. The C-SCAN schedule is 143, 913, 948, 1022, 1470, 1509, 1750, 1744, 4999, 0, 86, 130

$$D = |4999 - 143| + |4999 - 0| + |130 - 0| = 9985$$

21. 某磁盘的转速为 10 000 转/分，平均寻道时间是 6 ms，磁盘传输速率是 20 MB/s，磁盘控制器延迟为 0.2 ms，读取一个 4 KB 的扇区所需的平均时间约为

- A. 9 ms B. 9.4 ms C. 12 ms D. 12.4 ms

Outline

- **1 I/O Device**
- **2 I/O Controllers**
- **3 I/O Control Methods**
- **4 I/O Software Layer Structure**
- **5 I/O Application Interface**
- **6 The I/O Core Subsystem**
 - **6.1 Spooling Technology**
 - **6.2 Device Allocation & Recovery**
 - **6.3 Buffer Management**

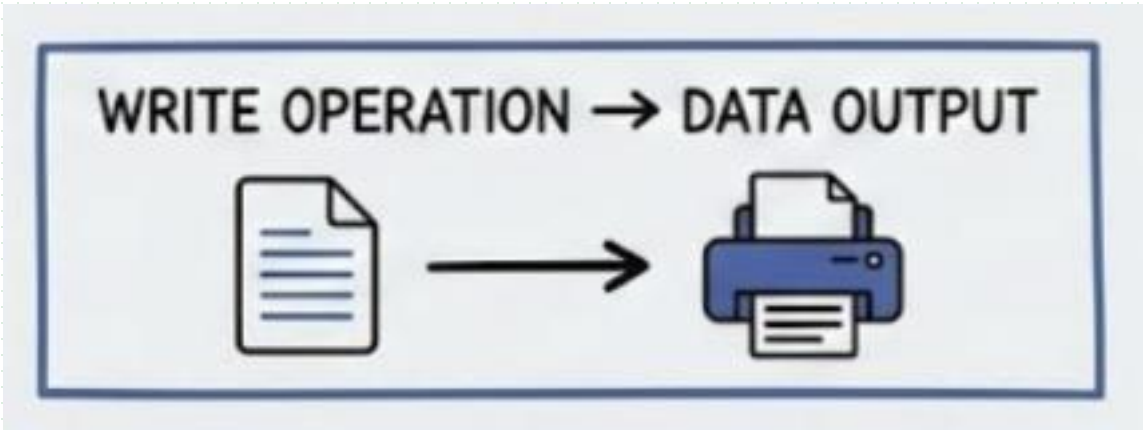
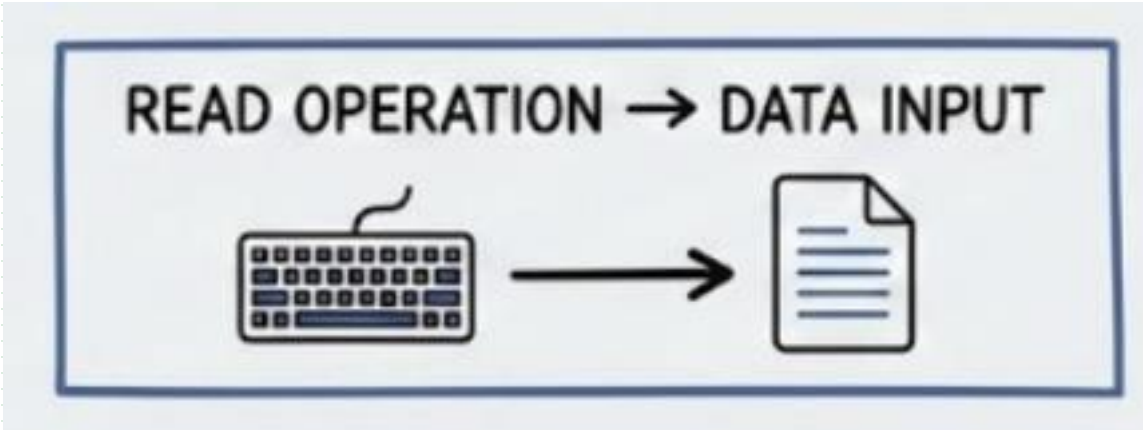
13.1 - What are I/O Devices?



I/O stands for Input / Output.

- Input: Devices that enter data into the computer (e.g., Keyboard, Mouse).
- Output: Devices that accept data from the computer (e.g., Monitor, Printer).
- Both: Some devices perform both functions (e.g., External Hard Drives, USB Flash Drives).

13.1 - From I/O to Read/Write



- UNIX systems abstract external devices as special files.
- Read Operation:
Reading from a file = Inputting data from a device.
- Write Operation:
Writing to a file = Outputting data to a device.

13.1 - Classification by Usage



- Human-Computer Interaction

Devices used to communicate with the user (Mouse, Keyboard, Printer).
Slow transmission speed.



- Storage Devices

Devices used to store data binaries (Hard Disks, Optical Disks).
Fast transmission speed.



- Network Devices

Devices for remote communication (Modems, Routers).
Medium transmission speed.

13.1 - Classification by Transmission Speed

Speed Category	Transmission Rate	Examples
Low Speed	Few to hundreds of bytes/sec	Mouse, Keyboard
Medium Speed	Thousands to tens of thousands of bytes/sec	Laser Printers
High Speed	Hundreds of thousands of bytes/sec +	Hard Disks, Optical Drives

*Note: Boundaries are not strict and serve as a general reference.

13.1 - Classification by Information Exchange Unit

■ Block Devices

Unit: Data is transferred in fixed-size Blocks.

Speed: High data transmission rate.

Addressable: Supports random access; can read/write any arbitrary block.

Examples: Hard Disk Drives, USB Drives.

■ Character Devices

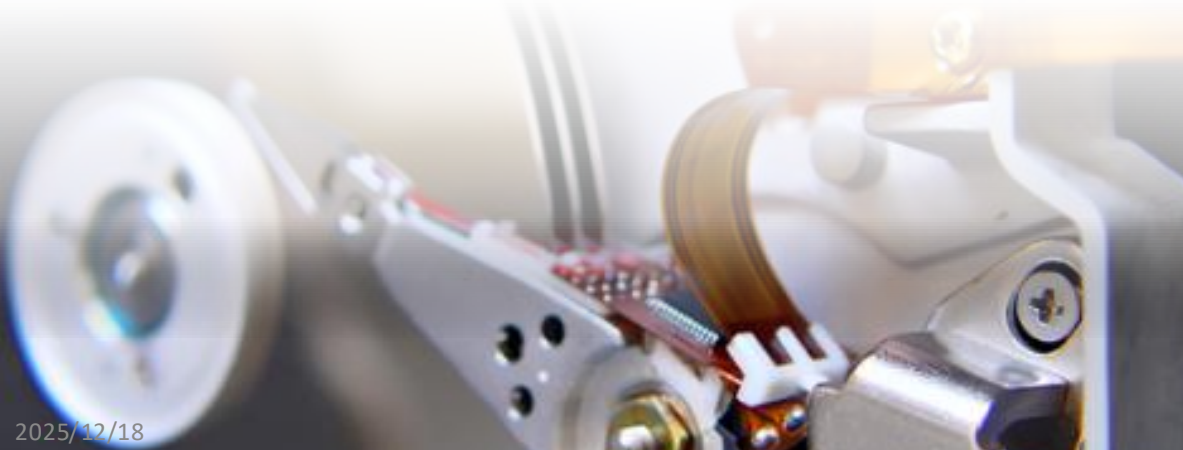
Unit: Data is transferred as a stream of Characters (bytes).

Speed: Low transmission speed.

Non-Addressable: Cannot seek to a specific position directly.

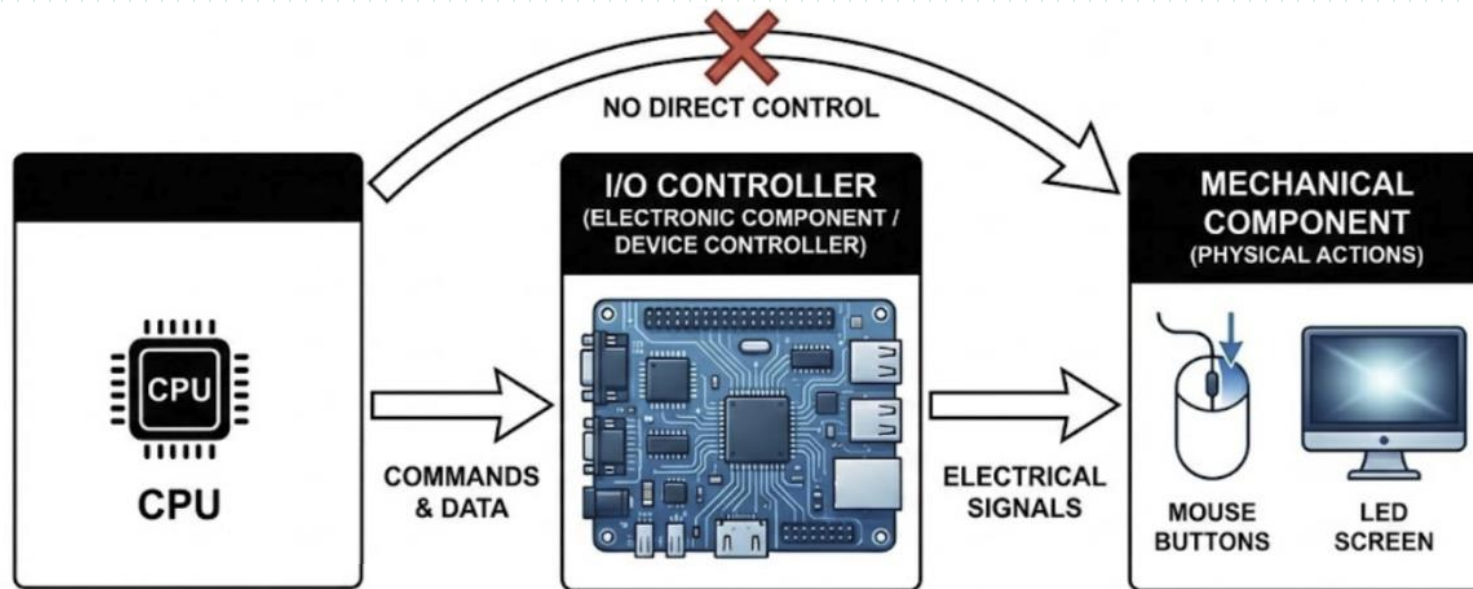
Control: Typically uses Interrupt-Driven I/O.

Examples: Mouse, Keyboard, Printers.



13.2 - I/O Controllers

- **Mechanical Components:** These are the visible, tangible parts of a device (e.g., mouse buttons, monitor screen) used to perform specific physical I/O operations.
- **Electronic Components:** This refers to the printed circuit board (PCB), also known as the I/O Controller or Device Controller.
- The Operating System focuses primarily on controlling these electronic components.



13.2 - The Role of the Controller

■ Function 1: Command Reception

The controller must receive and recognize commands sent by the CPU (e.g., Read, Write).

- ❑ Control Register: A specialized storage unit within the controller. The CPU writes commands and parameters (like read size) into this register. The controller executes operations based on the data stored here.

■ Function 2: Status Reporting

The CPU needs to know if a device is available before sending commands.

- ❑ Status Register: Stores the current state of the device (e.g., 1 = Idle, 0 = Busy, Error flags).The controller constantly updates this register. The CPU reads this register to make decisions, acting as a resource manager.

13.2 - The Role of the Controller

■ Function 3: Data Exchange

The controller acts as a data exchange buffer between the CPU and the device's mechanical parts.

- ▣ Data Register: Temporarily holds data being sent to the device (output) or received from the device (input). This buffering is essential because CPU speeds are significantly faster than mechanical device speeds.

■ Function 4: Address Recognition

An I/O controller contains multiple registers (Control, Status, Data), sometimes several of each.

To distinguish between them, each register is assigned a unique Address (similar to memory mapping).

The controller must identify which register the CPU intends to access based on the address signal provided on the bus.

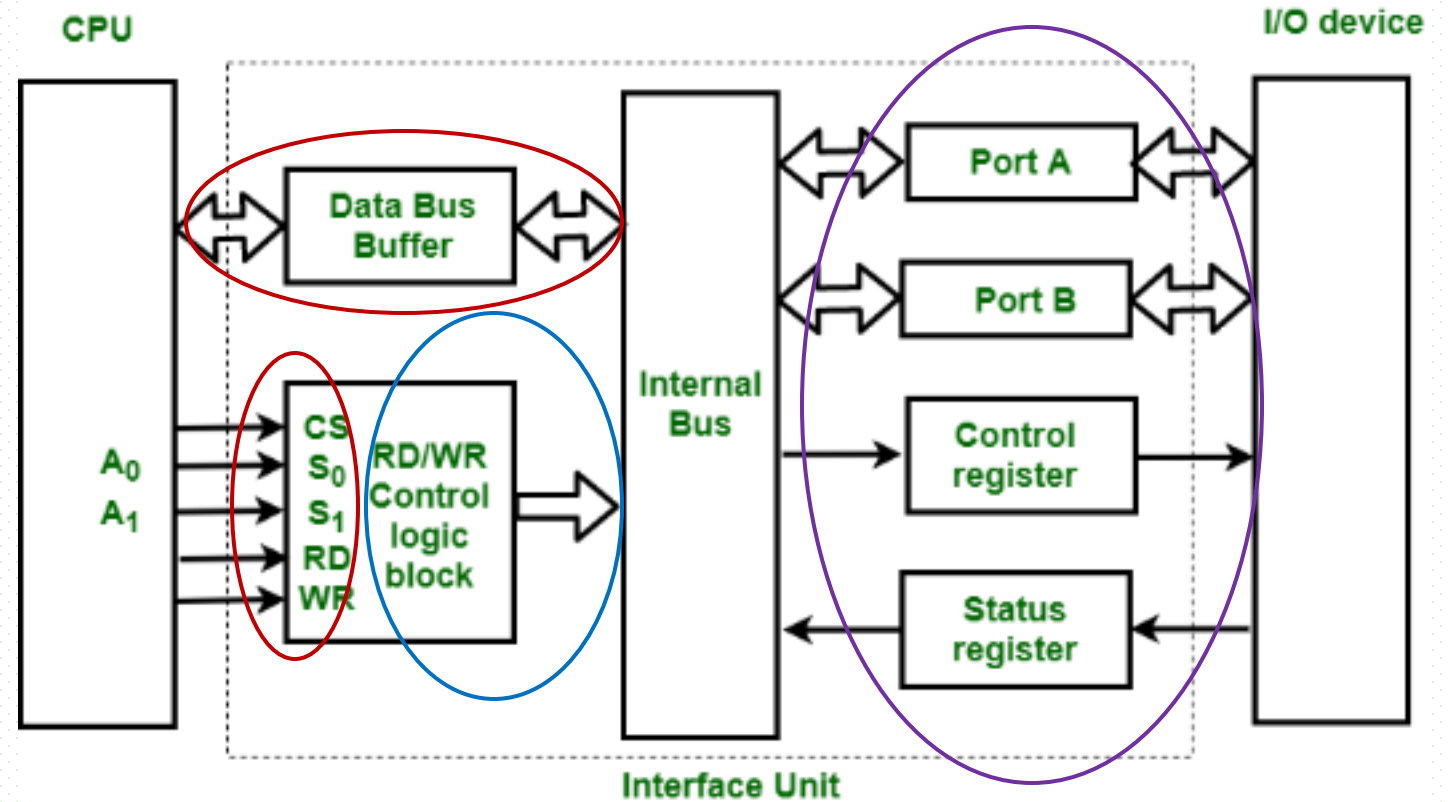
13.2 - Structure of an I/O Controller

- Generally, an I/O controller consists of three main parts:

1. CPU Interface: The connection point to the system bus (Decodes addresses, transfers data).

2. I/O Logic: The internal circuitry that processes commands and controls the device.

3. Device Interface: The connection to the physical mechanical components of the hardware.

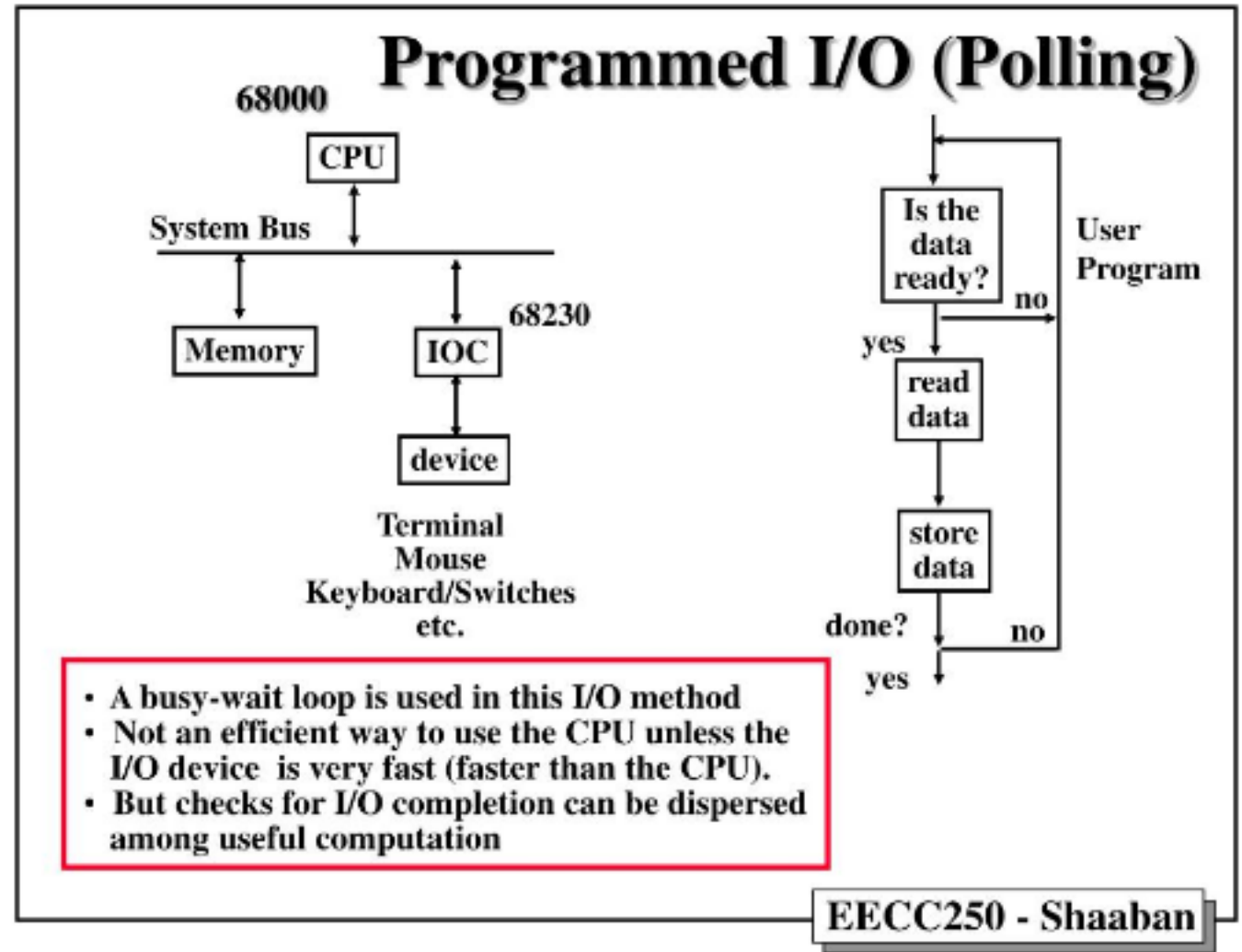


13.3 - I/O Control Methods

- 1. Programmed I/O (Polling)
- 2. Interrupt-Driven I/O
- 3. Direct Memory Access (DMA)
- 4. Channel Control

13.3 - Programmed I/O (Polling)

- The "Busy Wait" Approach
- This is the earliest and simplest method. The CPU sends a read command to the I/O controller and then continuously loops (polls), checking the status register.
- Key Mechanism: The CPU cannot do any other work while waiting. It constantly asks: "Are you ready? Are you ready?" until the device sets its status to 'Ready'.



13.3 - Programmed I/O (Polling)

■ Advantages

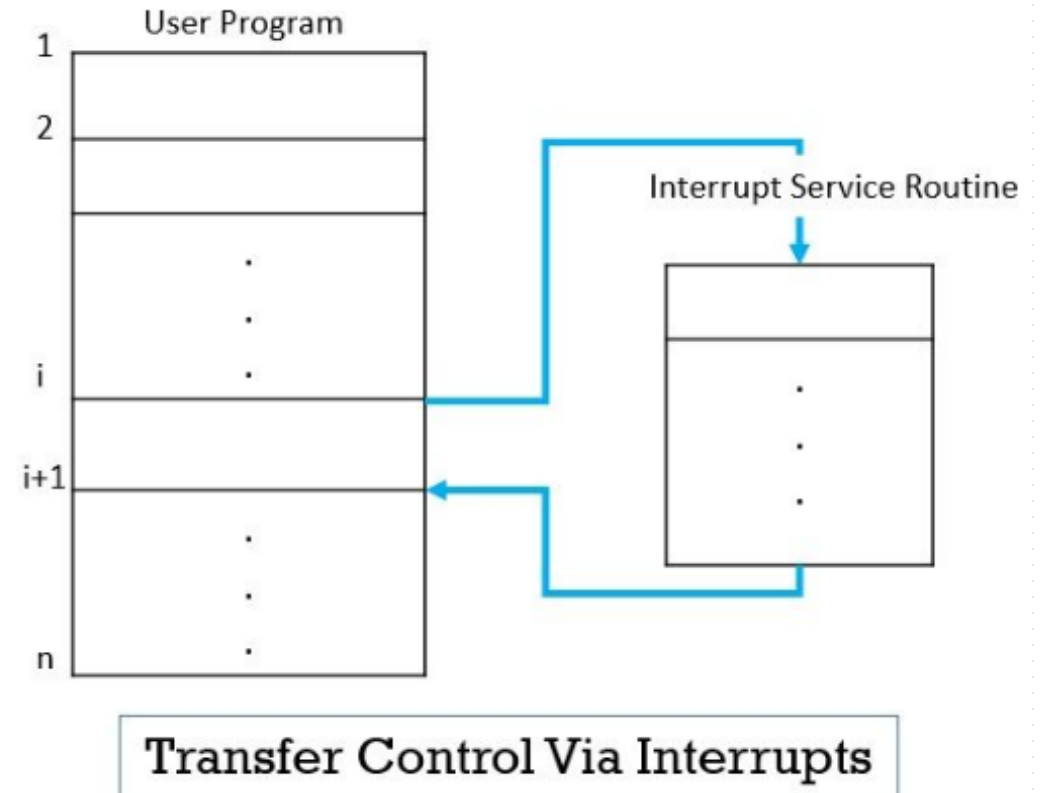
- ❑ Simplicity: It is very easy to implement in hardware and software. Requires minimal logic in the I/O controller.
- ❑ Control: The CPU has complete control over the entire process.

■ Disadvantages

- ❑ Low Efficiency: The CPU spends most of its time in a "Busy Wait" state, leading to extremely low CPU utilization.
- ❑ Serial Operation: The CPU and I/O device cannot work in parallel. One waits for the other.

13.3 - Interrupt-Driven I/O

- Wake Me Up When Ready
 - ❑ To solve the busy-wait problem, interrupts were introduced. The CPU issues an I/O command and immediately switches to executing other processes.
 - ❑ When the I/O device is finished preparing data, it sends an Interrupt Signal to the CPU. The CPU then pauses its current task to handle the data transfer.



13.3 - Interrupt-Driven I/O

- Advantages

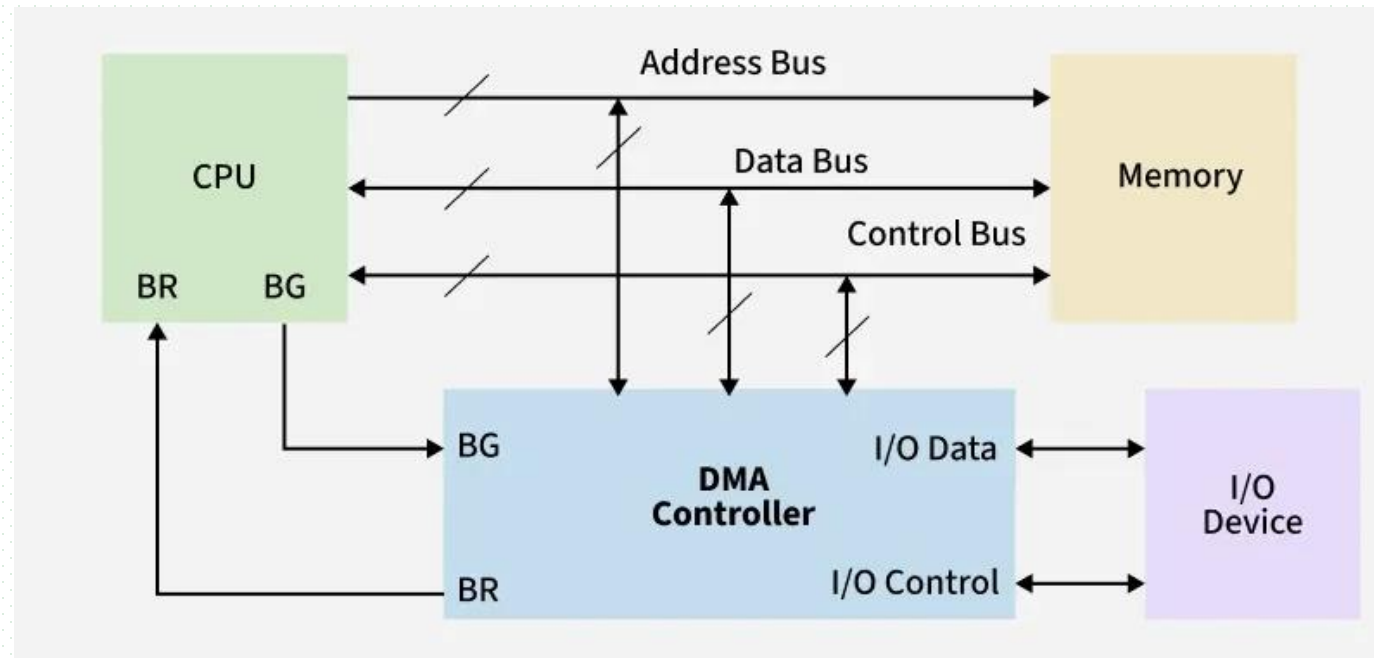
- Parallelism Achieved: The CPU and I/O device can finally work in parallel. The CPU processes other tasks while the I/O device is busy.

- Disadvantages

- Data Flow Limitation: Data must still pass through the CPU (Device → CPU Register → Memory).
- The Bottleneck: Interrupts occur per Word of data. Transferring a large file generates thousands of interrupts, creating significant CPU overhead for context switching.

13.3 - Direct Memory Access (DMA)

- Bypassing the CPU -- DMA was created to handle block transfers. A specialized DMA Controller takes over the bus control.
 - ▣ Mechanism: The CPU sets up the transfer (Start Address, Count) and lets the DMA controller handle the rest. Data flows directly from the I/O Device to Memory without passing through the CPU.
 - ▣ Result: An interrupt is generated only after an entire Block is transferred, not every word.



13.3 - Direct Memory Access (DMA)

- Advantages

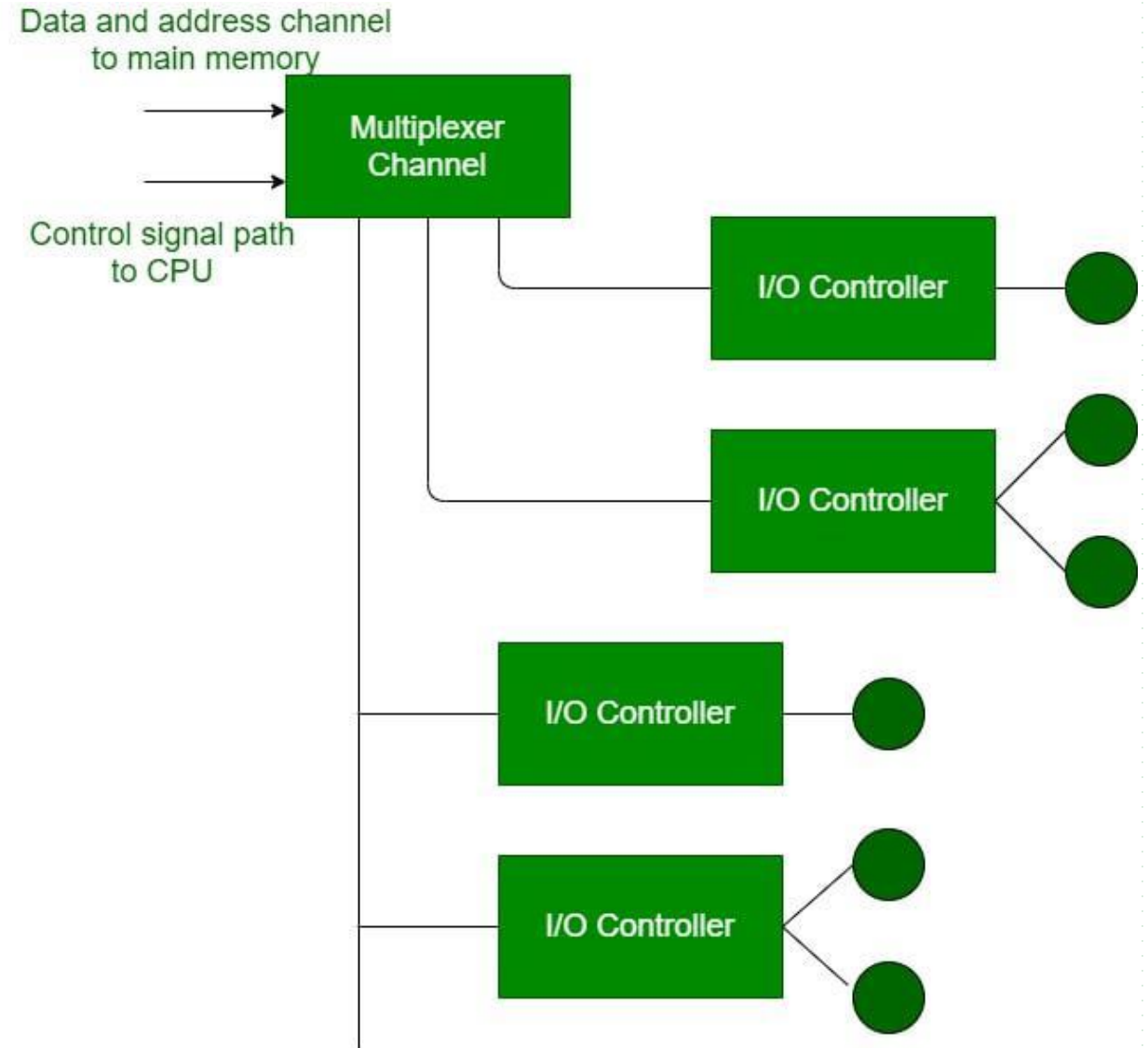
- ▣ Transfers one “block” rather than one “word”.

- Disadvantages

- ▣ It can only read and write contiguous blocks of data at a time. If the data we need to read is discretely distributed on the disk, or needs to be stored in a non-contiguous space in memory, the CPU must issue multiple DMA instructions, which is still not perfect.

13.3 - Channel Control

- The "Weak CPU"
- Channels are specialized processors designed specifically for I/O. They execute "Channel Programs" (task lists) stored in main memory.
- Advanced Capability: Unlike DMA which handles one continuous block, a Channel can handle multiple discrete blocks and control multiple I/O controllers simultaneously. The CPU just says "Execute this list" and walks away.



13.3 - Channel Control

- Advantages

- ▣ Transfers a group of “block” rather than single “block”.

- Disadvantages

- ▣ Hardware complexity.
- ▣ High cost.

13.3 - I/O Control Methods Comparison

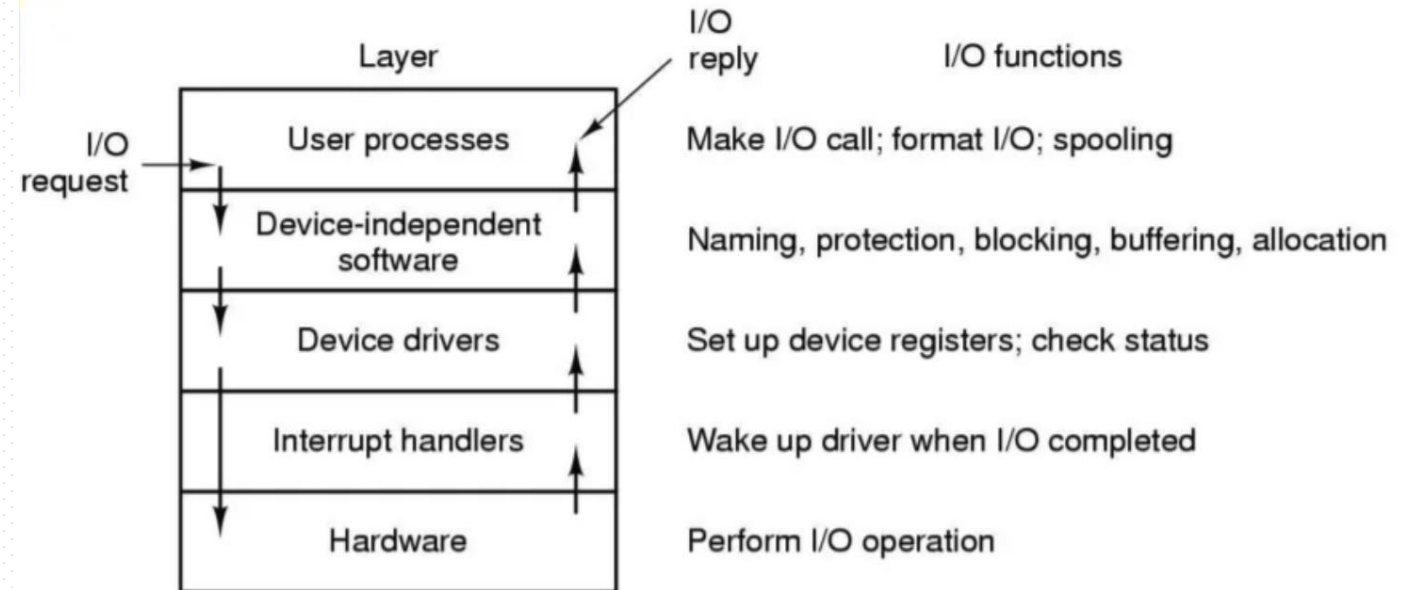
Control Method	Transfer Unit	Data Flow	CPU Intervention
Programmed I/O	Word	Device ↔ CPU ↔ RAM	Extremely High (100%)
Interrupt-Driven	Word	Device ↔ CPU ↔ RAM	High (Per Word)
DMA	Block	Device ↔ RAM	Low (Start/End of Block)
Channel Control	Group of Blocks	Device ↔ RAM	Lowest (Start/End of Program)

13.4 - I/O Software Layer Structure

■ The Layer Hierarchy

The I/O software structure consists of four distinct software layers sitting on top of the hardware. The closer to the bottom, the closer to the hardware.

- ❑ User-level I/O Software: Topmost layer, interacts with user.
- ❑ Device-Independent Software: System-wide uniform interfacing.
- ❑ Device Drivers: Specific to hardware manufacturers.
- ❑ Interrupt Handlers: Bottom software layer, handles signals.



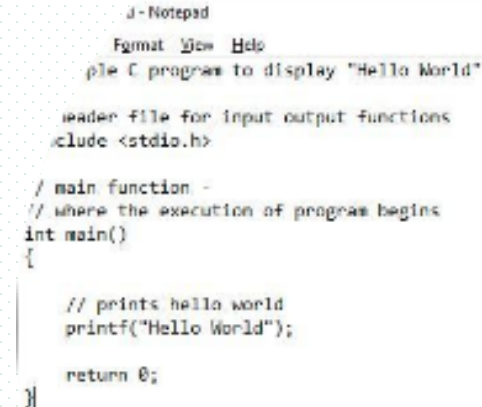
13.4 – Layer 1 - User-Level Software

- Simple Interfaces

User-level software provides simple, high-level libraries for interaction, such as C's `printf("Hello World")`.

- System Calls

These libraries translate high-level commands into system calls (e.g., `write`) to request Kernel services, as users cannot access hardware directly.



```

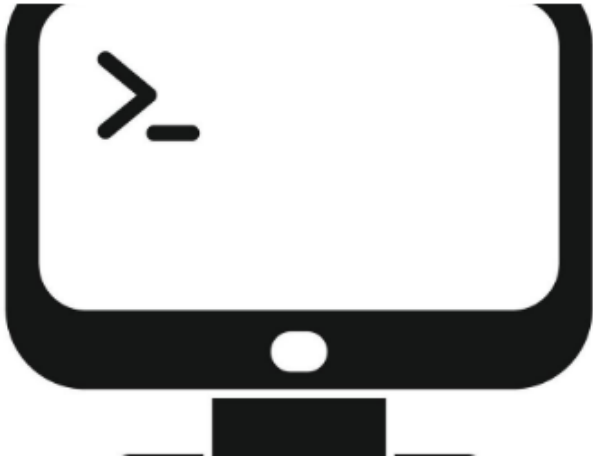
j - Notepad
Format View Help
File Edit Program to display "Hello World"

Header file for input output functions
#include <stdio.h>

// main function -
// where the execution of program begins
int main()
{
    // prints hello world
    printf("Hello World");

    return 0;
}
```

13.4 – Layer 2 - Device-Independent Software



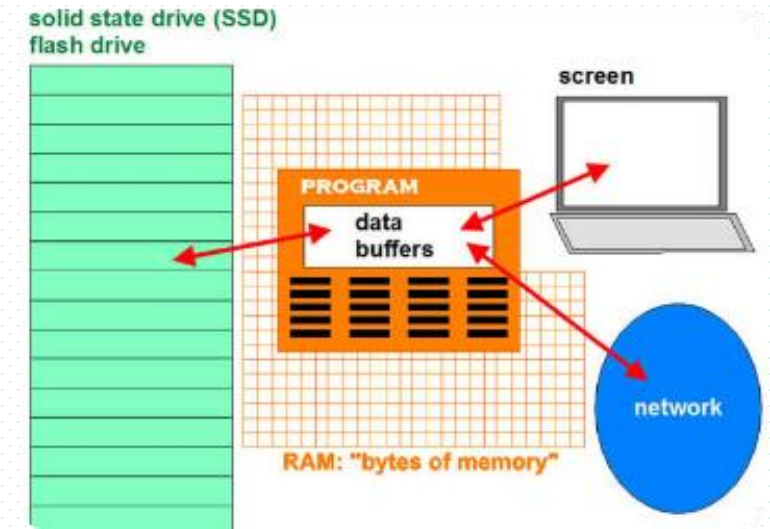
- Uniform Interface

Provides a standard naming convention and interface (e.g., read/write) regardless of the device type.



- Protection

Treats devices like files, managing access permissions to prevent unauthorized usage.



- Buffering

Manages data buffering to smooth out speed mismatches between the CPU and I/O devices.

13.4 – Layer 2 - Device-Independent Software

Logical Name (User View)	Physical Device (OS View)	Driver Entry Point
/dev/printer_01	Canon_LaserJet_X5	0x0045AF...
/dev/printer_02	HP_InkJet_Series	0x0088BB...
/dev/mouse	Logitech_USB_Receiver	0x0012CC...

The OS maps user-friendly names to physical hardware using a Logical Device Table (LUT).

13.4 – Layer 3 - Device Drivers

- Manufacturer Specific

Every device has different internal electronics (registers). Drivers are specific software provided by manufacturers to handle these differences.

- Hardware Abstraction

The driver hides the complex hardware details. It translates generic OS commands into specific bit-level operations required by the device controller.

13.4 – Layer 4 - Interrupt Handlers

- The First Responders

When hardware completes a task, it triggers an interrupt. The Interrupt Handler is the piece of software that catches this signal.

- Duties

It checks the device status (success or error), transfers data from the device to memory, and then passes control up to the device driver.

13.5 – I/O Application Interface

- The Problem

Applications need to access hardware via System Calls. However, devices are incredibly diverse (e.g., a Mouse vs. a Hard Disk).

- The Solution

The Device Independent Software layer provides unified interfaces (Character, Block, Network) to hide these hardware complexities from the user.

13.5 – Block Device Interface

- Example:
 - ▣ Hard Disk Drive
- Key Concepts:
 - ▣ Addressable: Data has specific locations (sectors).
 - ▣ Seekable: A read/write pointer can move to any address.
 - ▣ Bulk Access: Reads/Writes occur in blocks or multiple bytes.
- Primary Operations
 - ▣ `read/write(device, addr, size)`
 - ▣ `seek(device, offset)`



13.5 – Character Device Interface

- Example:
 - ▣ Keyboard, Printer, Mouse.
- Key Concepts:
 - ▣ Non-Addressable: Data flows as a continuous stream of characters.
- Primary Operations
 - ▣ `get()`: Read a single character.
 - ▣ `put()`: Write a single character.

Note: You cannot "seek" to a specific key on a keyboard.



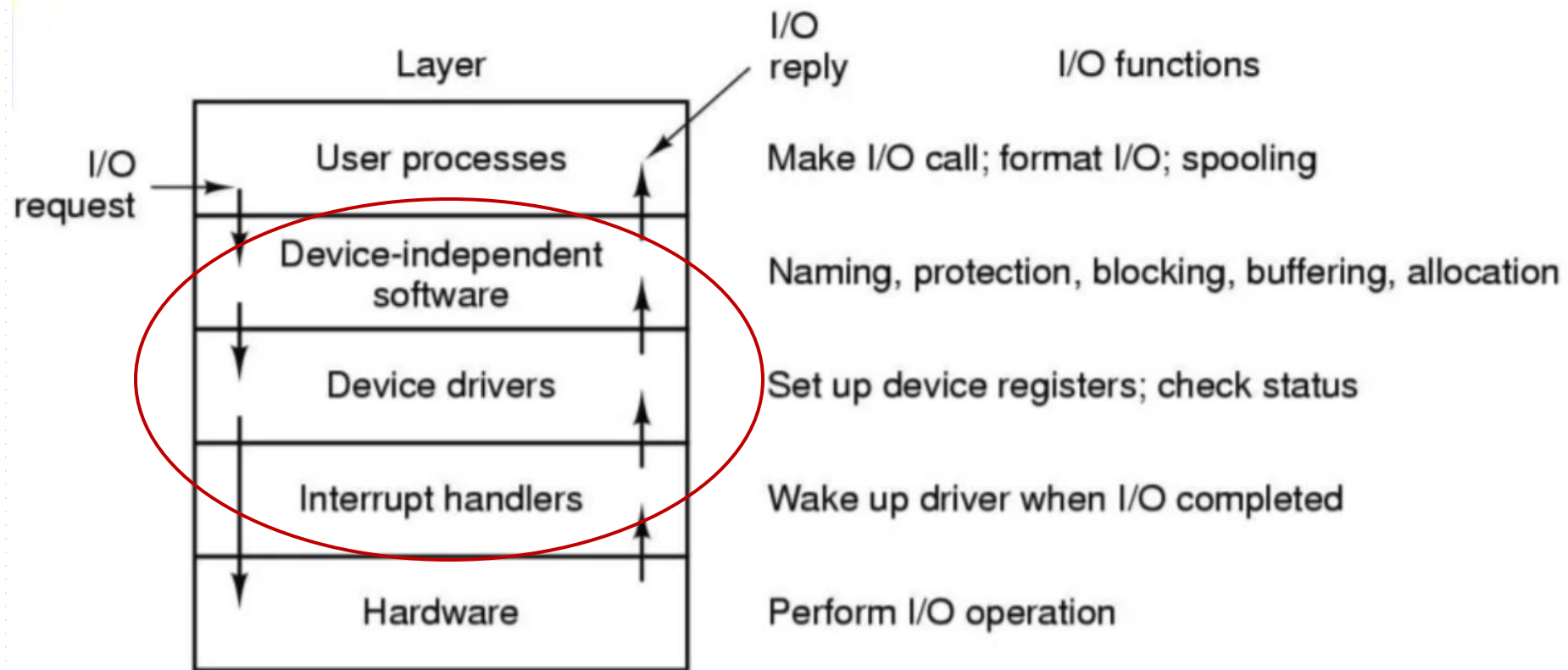
13.5 – Network Device Interface

- Example:
 - ▣ Network Card (NIC).
- Key Concepts:
 - ▣ Socket Abstraction: Data flows as a continuous stream of characters.
- Primary Operations
 - ▣ `socket()`: Creates a kernel-space buffer for communication.
 - ▣ `bind()`: Links the socket to a local Port.
 - ▣ `connect()`: Connects to a remote Host.
 - ▣ `read()/write()`



13.6 – I/O Core Subsystem

- I/O Core Subsystem
 - ▣ = Device-Independent Software + Device Drivers + Interrupt Handlers



13.6 – I/O Core Subsystem

■ I/O Core Subsystem Functions



I/O Scheduling

Ordering requests
for efficiency.



Device Protection

Managing access
permissions.



SPOOLing

Virtualizing
exclusive devices.



Allocation

Assigning and
reclaiming devices.



Buffering

Managing data flow
speed mismatch.

13.6 – Function 1 – I/O Scheduling

- Optimizing Order

Similar to CPU scheduling, I/O scheduling determines the optimal sequence for processing I/O requests to improve overall system efficiency.

- Common Algorithms

Techniques like First-Come-First-Serve (FCFS), SCAN, and C-SCAN algorithms are used. This is critical for disk drives to minimize mechanical seek time.

13.6 – Function 2 – Device Protection

- The "Device as File" Concept

In systems like UNIX, devices are treated as special files. This unifies the protection mechanism.

- File Control Block (FCB)

The system utilizes FCBs to track device permissions. When a user attempts to access a device, the kernel checks their read/write privileges against the FCB, exactly like standard file protection.

13.6.1 – Function 3 – Spooling Technology

- SPOOLing (Simultaneous Peripheral Operations On-Line)

In the early days of computing, input was handled by slow devices like paper tape machines. The CPU had to interact directly with these devices.

The Problem: The CPU is incredibly fast, while mechanical I/O is extremely slow. The CPU spent most of its time waiting for I/O to complete, wasting expensive computational resources.



13.6.1 – Function 3 – Spooling Technology

To solve the speed mismatch, "Offline Technology" was introduced using magnetic tape.

Programs were first loaded onto fast magnetic tapes using a separate satellite machine. The main CPU then read data from the high-speed tape, not the slow paper reader.

This decoupled the CPU from slow peripheral devices, significantly improving efficiency.



13.6.1 – Function 3 – Spooling Technology



Disk Storage

Input Well: Simulates the input tape.
Output Well: Simulates the output tape.



Memory Buffers

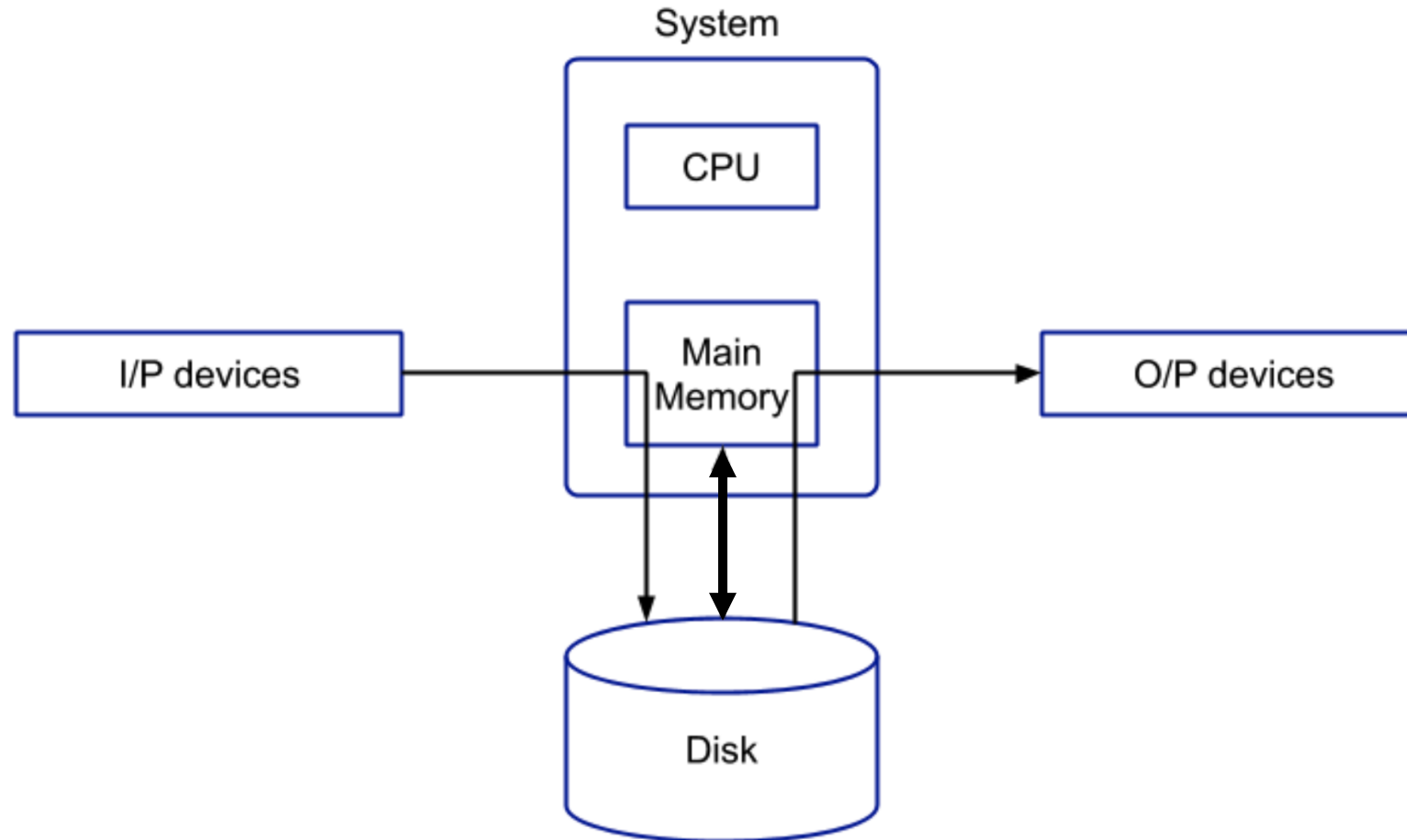
Temporary storage areas in RAM to facilitate fast data transfer between devices and the disk wells.



Processes

Input Process: Simulation of the input machine.
Output Process: Simulation of the output machine.

13.6.1 – Function 3 – Spooling Technology



Input: Device → Input Buffer A (Memory) → Input Well (Disk) → Input Buffer B (Memory) → CPU.

Output: CPU → Output Buffer A (Memory) → Output Well (Disk) → Output Buffer B (Memory) → Device.

13.6.2 – Function 4 – Device Allocation & Recovery

- Inherent Device Attributes
 - ▣ Exclusive Devices: Allocated to one process at a time (e.g., Printers). Requires waiting.
 - ▣ Shared Devices: Allocated to multiple processes macroscopically (e.g., Disk Drives).
 - ▣ Virtual Devices: Exclusive devices transformed into shared ones using SPOOLing technology.
- Allocation Algorithms
 - ▣ Standard scheduling strategies (e.g., FSFS).
- Safety Considerations
 - ▣ Deadlock

13.6.2 – Function 4 – Device Allocation & Recovery

Static Allocation

Allocate **all** required resources before the process begins execution.

- **Pros:** No Deadlock (Breaks "Hold and Wait").
- **Cons:** Low resource utilization; Process cannot start if one resource is missing.

Dynamic Allocation

Request resources **during** execution as needed.

- **Pros:** High efficiency; Resources used only when necessary.
- **Cons:** Risk of Deadlock (Requires Banker's Algorithm or detection mechanisms).

13.6.2 – Function 4 – Device Allocation & Recovery



DCT

**Device Control
Table**

Records device status, type, ID, and pointer to its Controller (COCT).



COCT

**Controller Control
Table**

Records controller status and pointer to its Channel (CHCT).



CHCT

**Channel Control
Table**

Records channel status and queue of waiting processes.



SDT

**System Device
Table**

System-wide list of all devices, pointers to DCTs and Drivers.

13.6.3 – Function 5 – Buffer Management

■ What is a Buffer?

A buffer is essentially a storage area used to hold data temporarily while it is being moved from one place to another.

- ▣ Hardware Buffers: Implemented using specialized hardware registers (e.g., TLB/Associative Registers). fast but expensive and small capacity.
- ▣ Memory Buffers: Implemented using a reserved section of RAM. This is the most common approach for Device-Independent Software.

13.6.3 – Function 5 – Buffer Management

- Function 1: Mitigating Speed Mismatch
 - ▣ Bridging the Gap: There is a significant speed difference between the CPU (very fast) and I/O devices (relatively slow).

- Function 2: Reducing Interrupt Frequency
 - ▣ Character-by-Character: Interrupt generated for every single character. High overhead.
 - ▣ Buffered: Interrupt generated only when the buffer is full (input) or empty (output).

- Function 3: Granularity
 - ▣ Solves mismatch in data size. A process may output data in large blocks, while an I/O device works with single characters. The buffer acts as an adapter between these sizes.

13.6.3 – Function 5 – Buffer Management

■ Single Buffering Strategy

The OS allocates one buffer in memory for the user process.

- ▣ T: Time to input one block of data into the buffer.
- ▣ M: Time to move data from buffer to user workspace.
- ▣ C: Time for the CPU to process/compute the data.

Constraint: The buffer cannot be written to while non-empty, and data can only be moved when the buffer is full.

13.6.3 – Function 5 – Buffer Management

To calculate the average time required to process one block of data:

Scenario A: $T > C$

Input time is slower than processing time.

$$\text{Time} = T + M$$

The CPU waits for the buffer to fill.

Scenario B: $T < C$

Input time is faster than processing time.

$$\text{Time} = C + M$$

The I/O device waits for the CPU to finish.

$$\text{Time} = \max(C, T) + M$$

13.6.3 – Function 5 – Buffer Management

- Double Buffering Strategy

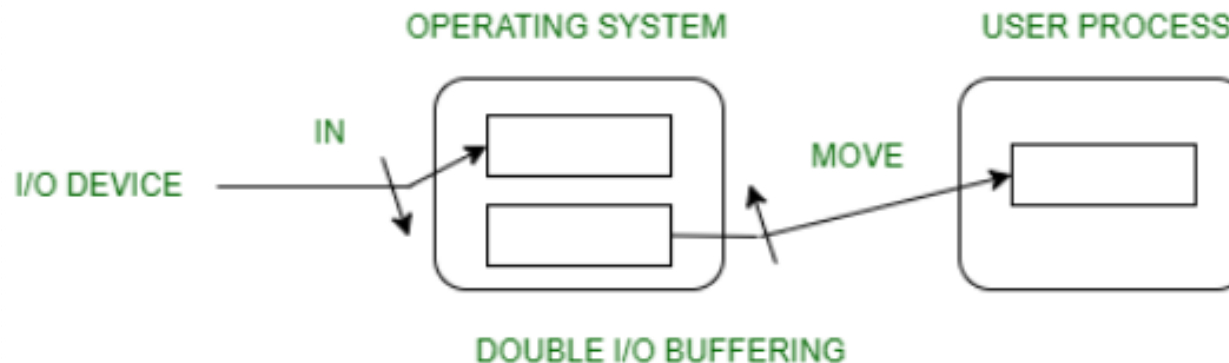
Uses two buffers to enable continuous data flow. Input fills Buffer 1 while CPU processes Buffer 2, then they swap.

- Average Time Per Block:

If $T > C+M$: Limited by Input (T)

If $T < C+M$: Limited by Processing ($C+M$)

$$\max(T, C + M)$$



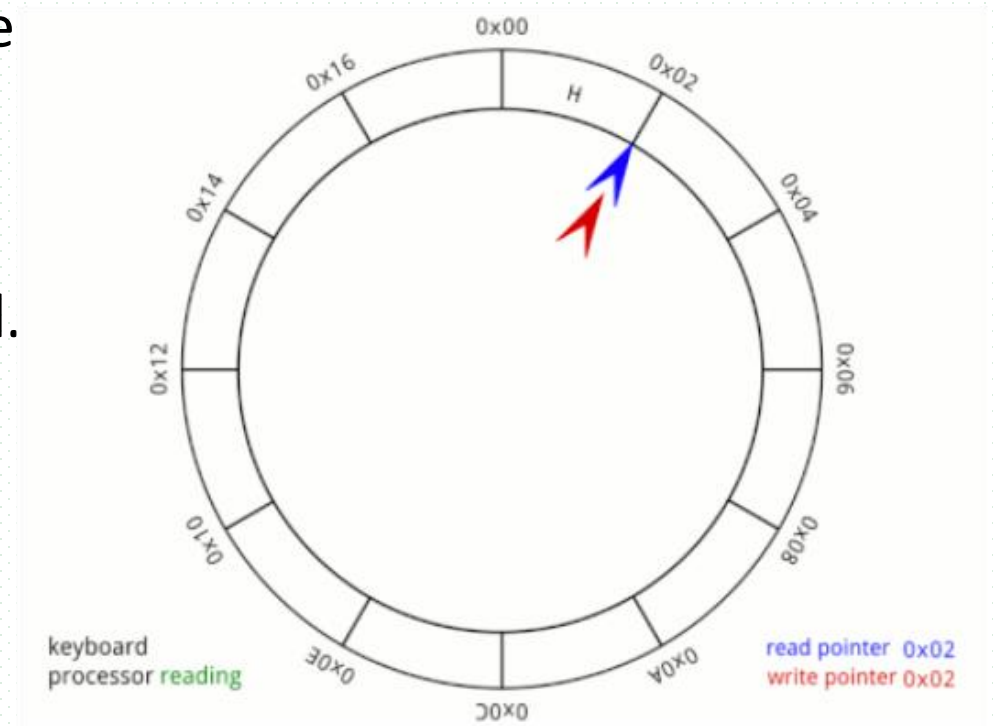
13.6.3 – Function 5 – Buffer Management

■ Circular Buffering

When two buffers aren't enough, multiple buffers are arranged in a ring.

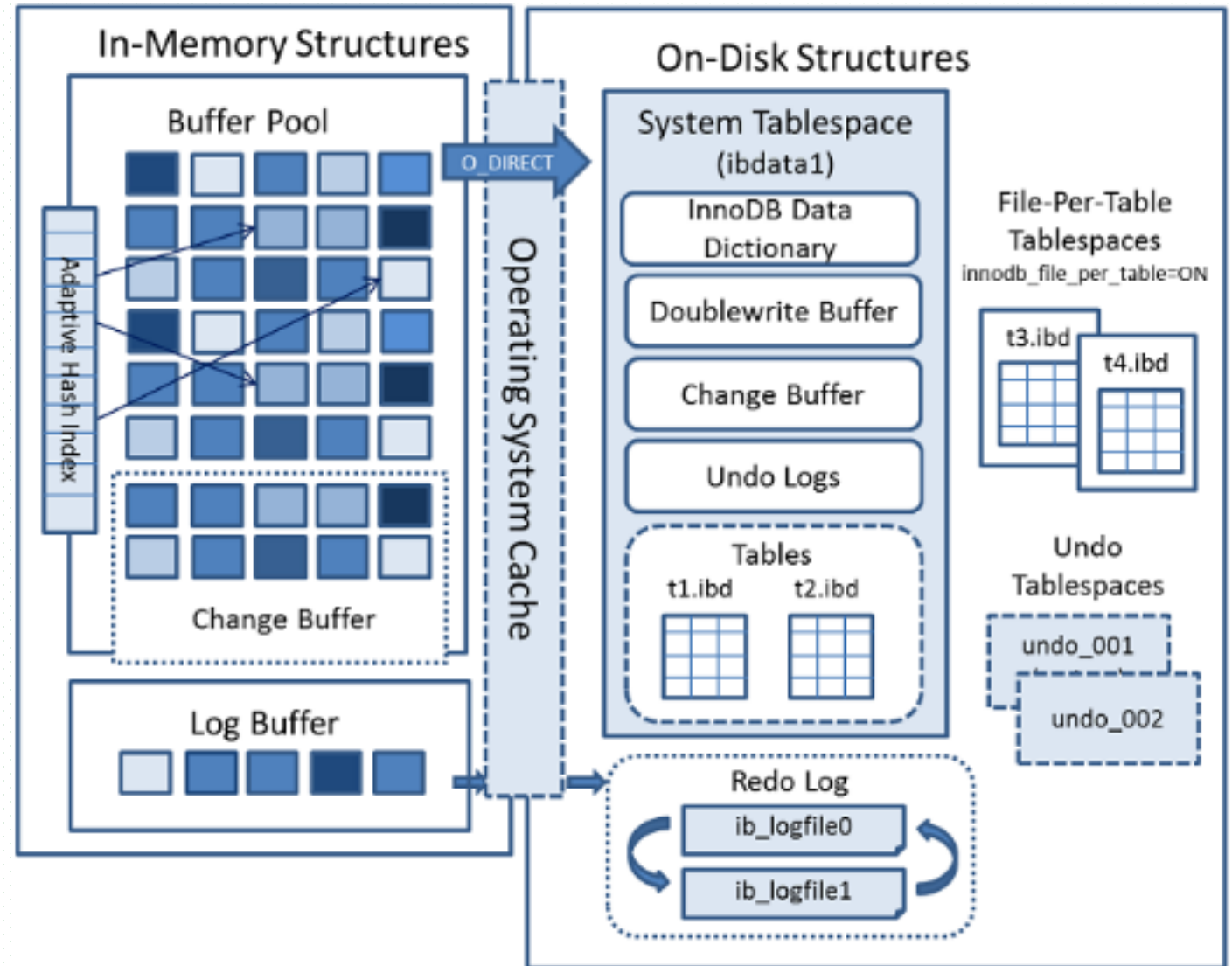
- ❑ Multiple Buffers: Ideal for bursty data traffic.
- ❑ IN Pointer: Points to the next empty buffer to fill.
- ❑ OUT Pointer: Points to the next full buffer to empty.

This structure allows the system to absorb rapid bursts of input without slowing down processing.



13.6.3 – Function 5 – Buffer Management

- Buffer Pool
- A unified collection of buffers shared across the system.
- 3 Queues: Empty Queue, Input Queue (Full), Output Queue (Full).
- 4 Work Buffers:
 - hin: Receive Input
 - sin: Extract Input
 - hout: Receive Output
 - sout: Extract Output





Thanks for your
attention



北京邮电大学